

@skills: ATTENTION IS ALL YOU HAVE

<https://arxiv.org/pdf/2608.12610>

1. Problem

- 문제
 - 현재 Agent Skill은 대부분 `install` 방식으로 배포됨.
 - 설치하면 Skill의 설명이 매 요청마다 system prompt에 상주함.
 - 2026년 7월 기준 56,804개의 Skill이 존재하지만, 실제로 안정적으로 자동 호출할 수 있는 attention budget은 100개 미만으로 추정됨.
 - 사용하지 않는 Skill까지 context를 차지하면서 필요한 Skill의 활용도까지 떨어지는 문제.
- 왜 중요한가?
 - Skill이 많아질수록 prompt가 비대해지고, attention이 분산되어 LLM의 instruction following과 Skill triggering이 악화될 수 있음.

2. Method

- 핵심 아이디어: `@skills` 프로토콜
 - 기존 `install = 가져오기 + 저장 + 자동 호출` 을 세 가지 기능으로 분리:
 - Reference → 필요할 때 읽기
 - Save → 프로젝트에 복사해서 관리
 - Install → 정말 필요한 Skill만 자동 호출 대상으로 등록
- 핵심 구조
 - `@skills:<path>` → Skill을 필요한 순간에만 읽음
 - `:save` → Skill을 Git-tracked 프로젝트에 저장하여 수정·관리
 - `:install` → 자동 triggering에 필요한 정보만 prompt에 상주시킴

- Skill의 본문은 기본적으로 prompt에 상주하지 않음.
- 기존 방법과 차이
 - 기존: 모든 Skill을 설치 → 항상 description을 context에 유지
 - @skills: 사용 빈도와 자동 호출 필요성에 따라 context residency를 선택
 - 즉, "Skill을 설치해야 사용할 수 있다" → "필요할 때 불러서 사용한다"

3. Experiments & Results

- 실험 성격
 - 전통적인 benchmark 기반 성능 비교 논문은 아님.
 - 2026년 7월 공개 Skill 생태계를 분석:
 - 56,804개 Skill
 - 1,133개 GitHub repository
 - 56,245개 Skill body를 실제 수집·분석
 - 일부 분류 분석은 100개 Skill을 uniform sample로 사용
- 주요 관찰
 - 설치된 Skill 하나당 약 50~280 tokens가 매 요청마다 발생.
 - 한 실제 작업 환경에서는 18개 Skill이 약 1,500~2,000 tokens의 상시 overhead를 발생시킴.
 - 일부 Skill은 trigger 조건을 과도하게 작성해 일반적인 description보다 약 20배 많은 token을 사용하는 경우도 관찰됨.
- 중요한 점
 - @skills가 실제 task 성능을 몇 % 향상시킨다는 직접적인 benchmark 결과는 없음.
 - 설치된 Skill 수에 따른 trigger reliability를 통제 실험하지 않았음.

4. Insight

- 핵심 기여 한 줄
 - Agent Skill을 "항상 prompt에 설치해 두는 지식"에서 "필요할 때 검색·호출하는 외부 메모리"로 바꾸자는 제안.
- 한계점

- 실제 LLM task 성능이나 Skill 호출 정확도를 얼마나 향상시키는지 검증한 controlled experiment가 없음.
- "100개 미만의 reliable auto-trigger slots" 역시 직접 측정한 값이 아니라 기존 연구와 관찰을 바탕으로 한 추정임.
- 저자가 해당 프로토콜과 CLI/Hub를 직접 개발하고 있어 이해관계가 존재함.
- 내가 가져갈 포인트
 - Context Window가 크다고 해서 모든 정보를 넣는 것이 좋은 것은 아님.
 - Agent 시스템에서는 "무엇을 알고 있는가"보다 "언제 context에 넣을 것인가"가 중요함.
 - RAG와 비슷한 구조로 이해할 수 있음.
 - RAG → 필요한 지식을 그때 가져옴
 - @skills → 필요한 행동 지침이나 절차를 그때 가져옴
 - Agent 설계에서 Just-in-Time Context, Progressive Disclosure, External Memory가 중요한 설계 원칙이 될 수 있음.